

一 SQLang: Efficient Execution of Structured Language Model Programs (NeurIPS 2024)

核心问题: LLM 应用越来越多地涉及多轮生成、结构化输入输出和控制流,但现有推理系统缺乏对这类复杂程序的编程抽象和执行优化。

主要贡献: SQLang 提出了一套前端语言与运行时系统。前端提供生成与并行控制原语,简化复杂 LLM 程序的编写;运行时通过两项关键优化加速执行:

RadixAttention: 自动 KV Cache 复用。请求完成后不丢弃 KV Cache,而是将 prompt 和生成结果的 KV 存入一棵基数树 (radix tree),以支持高效的前缀搜索、复用、插入和淘汰。配合 LRU 淘汰策略和 cache-aware 调度,显著提升缓存命中率。在共享系统提示、多轮对话、few-shot 等场景下效果尤为突出。

压缩有限状态机加速结构化输出解码,适用于 JSON、正则约束等场景。

实验结论: 在 agent 控制、逻辑推理、JSON 解码、RAG 多轮对话等任务上,吞吐量相比当时最优推理系统提升最高 6.4。

思考: RadixAttention 的本质是将 KV Cache 管理从“请求级”提升到“程序级”,利用程序内在的前缀共享结构来避免重复计算。这是一种典型的“利用工作负载结构换取效率”的系统设计思路。

二 Efficient Memory Management for Large Language Model Serving with PagedAttention (SOSP 2023)

核心问题: LLM 服务中 KV Cache 显存占用巨大且动态变化,传统实现为每个

请求预分配连续的最大序列长度内存,导致严重的内部碎片约 60% 浪费和外部碎片,限制了 batch size。

主要贡献: PagedAttention 借鉴操作系统虚拟内存分页机制,将 KV Cache 划分为固定大小的 block (通常 16 或 32 token),按需分配,物理上无需连续存储。每个序列维护一张 block table,记录逻辑 block 到物理 block 的映射。在此基础上构建的 vLLM 服务系统实现了: (1) KV Cache 显存浪费接近零 (从 ~60% 降至 <4%) (2) 支持请求内部和请求之间的 KV Cache 灵活共享。

实验结论: 相比 FasterTransformer 和 Orca, vLLM 在相同延迟下吞吐量提升 2.4 倍,且序列越长、模型越大、解码算法越复杂,提升越显著。

思考: PagedAttention 的核心洞察是“注意力计算不需要连续的 KV 内存”。将分页抽象引入 GPU 显存管理,本质上是用“地址间接映射”的少量开销换取显存利用率的巨大提升。这与操作系统中虚拟内存解决物理内存碎片问题的思路一脉相承。